

SRS

Court Case Management

Guide Name: Shine K George

Name: Erfan Habeeb

Riya Dominic

Date:

Batch : MCA 3A

Table of Contents

1.INTRODUCTION	3
2. PROBLEM STATEMENT	3
3. SCOPE AND RELEVANCE OF THE PROJECT	3
4. OBJECTIVES	3
5. SYSTEM ANALYSIS	4
Introduction	4
6. EXISTING SYSTEM	4
7. LIMITATIONS OF EXISTING SYSTEM	4
8. PROPOSED SYSTEM	4
9. ADVANTAGES OF PROPOSED SYSTEM	5
10. FEASIBILITY STUDY	5
Technical Feasibility	5
Operational Feasibility	5
Economic Feasibility	5
11. SOFTWARE ENGINEERING PARADIGM APPLIED	6
12. SYSTEM ENVIRONMENT	6
Software Requirements	6
Hardware Requirements	6
Tools & Platforms	6
13. USER CHARACTERISTICS	7
14. SUMMARY OF SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS	7
14.1 User Authentication and Role Management	7
14.2 Case Filing By Client	8
14.3 Case Assignment By Admin	8
14.4 Hearing and Judgement Updates By Judge	9
14.5 Case search and filter	9
14.6 Document Upload and Download.....	10

14.7 Dashboard and Analytics.....	10
-----------------------------------	----

1. INTRODUCTION

This Software Requirement Specification (SRS) defines the functional and non-functional requirements for the Court Case Management System.

The system aims to address inefficiencies in the current manual process of handling court cases by introducing a digital platform that simplifies the process of case filing, assignment, document submission, hearing tracking, and judgement updates.

2. PROBLEM STATEMENT

The current legal system relies heavily on manual operations, including case filing, hearing schedules, lawyer assignments, and document handling.

This process is time-consuming, paper-based, and prone to errors. Clients must visit courts physically for updates, and case histories are often scattered or lost due to poor record management.

There is no centralized platform where all stakeholders — clients, lawyers, and court clerks — can collaborate and access information in real-time.

The absence of digital support leads to delays, miscommunication, and an overload of administrative work.

This project aims to solve these issues by developing a web-based system that streamlines and automates the court case handling process.

3. SCOPE AND RELEVANCE OF THE PROJECT

The Court Case Management System aims to simplify and automate key legal processes such as case filing, hearing scheduling, and case status updates. It allows clients to submit cases online, enables admins to assign judges and lawyers, and provides lawyers with tools to manage and update their assigned cases. The system also supports document uploads and real-time case tracking through role-based dashboards.

While the system focuses on internal court operations, it does not include integration with official government judicial databases or biometric authentication. Its relevance lies in promoting digital transformation within local courts and legal service providers, helping reduce manual work and improve efficiency.

4. OBJECTIVES

The primary objective of this project is to deliver a digital platform that improves the operational efficiency of court-related processes. From a business perspective, the system offers significant benefits such as reduced administrative workload, minimized paperwork, and faster case handling. It helps legal organizations and court branches lower operational costs by automating repetitive tasks like case assignment, hearing schedules, and status tracking. Additionally, it enhances service delivery by enabling clients to access case information remotely, improving client satisfaction and reducing in-person inquiries. For legal professionals and administrators, the system offers a streamlined workflow, centralized access to documents, and data-driven insights through dashboard analytics, all of which contribute to better decision-making and resource management.

5. SYSTEM ANALYSIS

The Court Case Management System is structured to handle various legal tasks based on user roles. The system operates using a modular approach, where each module addresses specific functionalities such as user authentication, case management, hearing schedules, document uploads, and reporting. Clients interact primarily with the case filing and tracking module, while lawyers access assigned cases and update hearing details. Admin users oversee the assignment of cases and maintain court schedules. The backend uses RESTful APIs built in Spring Boot, connected to a MySQL database, and the frontend uses React for a responsive interface. The system's modular design ensures scalability, maintainability, and separation of concerns across user responsibilities.

6. EXISTING SYSTEM

Currently, court systems rely heavily on manual procedures for managing legal cases. Clients must physically visit court offices to file a case or check the status of an ongoing case. Court clerks maintain case details in physical files or isolated software with limited functionality. Lawyers receive updates either verbally or through printed notices, making the process time-consuming and inefficient. There is no central digital repository for documents, no role-based access, and no automation in scheduling or tracking. These limitations slow down the delivery of justice and increase operational overhead.

7. LIMITATIONS OF EXISTING SYSTEM

- Prone to data loss and delays
- Requires physical presence for case status
- No centralized access or search
- Lacks proper workload distribution

8. PROPOSED SYSTEM

The proposed system addresses the limitations of the current setup by offering a secure, centralized, and user-friendly digital platform. It allows clients to file cases online, view updates, and access documents at any time. Admin users can assign cases, manage schedules, and generate reports through an intuitive dashboard. Judge(user) can update case hearings. Lawyers can view assigned tasks, and access digital files easily. The system automates notifications, simplifies workload distribution, and supports fast, secure document handling. By digitizing the workflow, the proposed system ensures faster, more transparent, and error-free management of court proceedings.

9. ADVANTAGES OF PROPOSED SYSTEM

- Centralized and secure system
- Easy access to all case information
- Improved workflow and transparency
- Faster document handling and updates

10. FEASIBILITY STUDY

10.1 Technical Feasibility

The project is technically feasible with the use of modern, well-supported technologies. The frontend will be developed using React.js, which offers a dynamic and responsive interface. The backend is built using Spring Boot, known for its robustness and scalability. Communication between the frontend and backend is handled via RESTful APIs, and the system will use MySQL for structured data storage. All of these technologies are open-source and widely used, ensuring sufficient resources, documentation, and community support for smooth development and maintenance.

10.2 Operational Feasibility

The proposed system aligns well with the operational needs of local courts, law firms, and legal service providers. Its role-based interface makes it user-friendly for clients, lawyers, and judge with minimal training. Since the system simplifies tasks like case filing, assignment, and tracking, it fits naturally into existing workflows while improving efficiency. The system can be adopted in stages and scaled based on user requirements, reducing operational risk during rollout.

10.3 Economic Feasibility

The system is economically viable due to its reliance on open-source tools and frameworks, eliminating software licensing costs. Hosting, deployment, and maintenance expenses are minimal and manageable. By reducing administrative tasks and manual labor, the system also

offers long-term cost savings. Faster case processing and reduced paper usage translate to improved productivity and reduced overhead for the organization implementing the system.

11. SOFTWARE ENGINEERING PARADIGM APPLIED

For this project, the Incremental Model is used. This model allows the system to be developed and delivered in parts (increments), where each increment adds functional capabilities. Initially, core modules like user login and case filing are developed. In subsequent increments, features such as hearing schedules, lawyer assignments, and dashboard insights are added. This model allows early testing and user feedback, making it easier to make adjustments along the way. It also reduces risks and ensures better progress tracking throughout development.

12. SYSTEM ENVIRONMENT

12.1 Software Requirements Specification

- **Frontend:** React.js, Axios, React Router
- **Backend:** Spring Boot, Spring Security, Spring Data JPA
- **Database:** MySQL
- **Authentication:** JWT (JSON Web Token), TOTP (Time Based One Time Password)
- **API Tool:** Postman for testing REST APIs

12.2 Hardware Requirements Specification

- **Processor:** Intel i5 or equivalent (or higher)
- **RAM:** Minimum 4 GB
- **Storage:** 250 GB HDD or SSD
- **Display:** 1366x768 resolution or higher
- **Internet:** Stable broadband connection for cloud or server access

12.3 Tools and Platforms

- **IDEs:** Visual Studio Code (Frontend), IntelliJ IDEA (Backend)
- **Operating System:** Windows 10/Linux
- **Browser:** Google Chrome or Mozilla Firefox
- **UML Tool:** draw.io for diagrams
- **Version Control:** Git and GitHub

13. USER CHARACTERISTICS

The system is designed for the following user roles:

- **Admin:** Typically a court assistant or office staff with basic technical skills. They manage case assignments, scheduling, and user roles, update hearing details.
- **Lawyer:** Legal professionals who need access to their assigned cases. They should be able to view documents, and check schedules.
- **Client:** General users with minimal technical background. The interface for clients is designed to be simple and intuitive for easy case filing and tracking.
- **Judge:** can update case hearing.

14. SUMMARY OF SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS

14.1 User Authentication and Role Management

Introduction:

Enables secure login for different users — Admin, Lawyer, Judge and Client — and provides access to role-specific features.

Inputs:

Username, Password, User Role (selected during registration or predefined by admin), 2FA CODE

Processing:

- Validate user credentials using Spring Security and JWT
- Check role and generate access token

- verify with 2FA From TOTP app
- Redirect to respective dashboard

Outputs:

- Successful login and dashboard access
- Error message for invalid login

14.2 Case Filing by Client**Introduction:**

Allows clients to digitally file a case by entering details and uploading supporting documents.

Inputs:

Case title, Case type (civil/criminal), Description, Client details, Attachments (PDF/Image)

Processing:

- Store case information in database
- Assign case a unique case ID
- Upload documents and link to case

Outputs:

- Confirmation message with case ID
- Case listed under client's dashboard

14.3 Case Assignment by Admin**Introduction:**

Admin assigns the case to a lawyer and judge, and sets an initial hearing date.

Inputs:

Case ID, Lawyer ID, Judge ID, First hearing date

Processing:

- Update case status to “Assigned”
- Store linked lawyer and judge IDs
- Schedule and store hearing data

Outputs:

- Updated case listing for lawyer and judge
- Email/notification (optional)

14.4 Hearing and Judgment Updates (Judge)**Introduction:**

The Judge updates the case after hearings and adds summaries or verdicts.

Inputs:

Case ID, Hearing summary, Date, Optional: next hearing date, Verdict

Processing:

- Append hearing details to case history
- Update case status if required (e.g., Ongoing → Closed)

Outputs:

- Updated hearing log in case record
- Reflected in client and admin dashboards

14.5 Case Search and Filter**Introduction:**

Allows users to search and filter cases based on multiple parameters.

Inputs:

Case ID, Client name, Case type, Date range, Status

Processing:

- Query database using provided filters
- Sort and display matching results

Outputs:

- Filtered case list with basic case details

14.6 Document Upload and Download**Introduction:**

Supports uploading evidence and downloading judgments or summaries.

Inputs:

Document file (PDF/Image), Case ID

Processing:

- Validate file type and size
- Store file securely linked to the case

Outputs:

- Confirmation of successful upload
- File appears in download section for all linked users

14.7 Dashboard and Analytics (Admin)**Introduction:**

Shows total cases, breakdowns by type/status, and workload distribution.

Inputs:

No user input — system-generated from case database

Processing:

- Count and group cases by type, status, role, or month
- Generate chart data

Outputs:

- Pie charts, bar graphs, and stats on admin dashboard
- Printable/downloadable reports

14.8 Lawyer Functional Requirements

Introduction:

The Lawyer interacts with the system to manage and follow up on cases assigned to them. Lawyers focus on reviewing assigned cases, preparing for hearings, and collaborating with clients through case documents.

Inputs:

- Case ID (from assigned cases)
- Optional: Notes or remarks for case preparation
- Uploaded documents (if required)

Processing:

- Retrieve and display all cases assigned by Admin
- Allow lawyer to upload supporting documents or legal notes
- Enable lawyer to view uploaded case materials, hearing schedules, and judge updates
- Provide search and filter functionality to quickly access case details

Outputs:

- Updated case dashboard for the lawyer with all assigned cases
- Confirmation of uploaded supporting documents
- Notifications when new hearings or judgments are updated by the Judge

